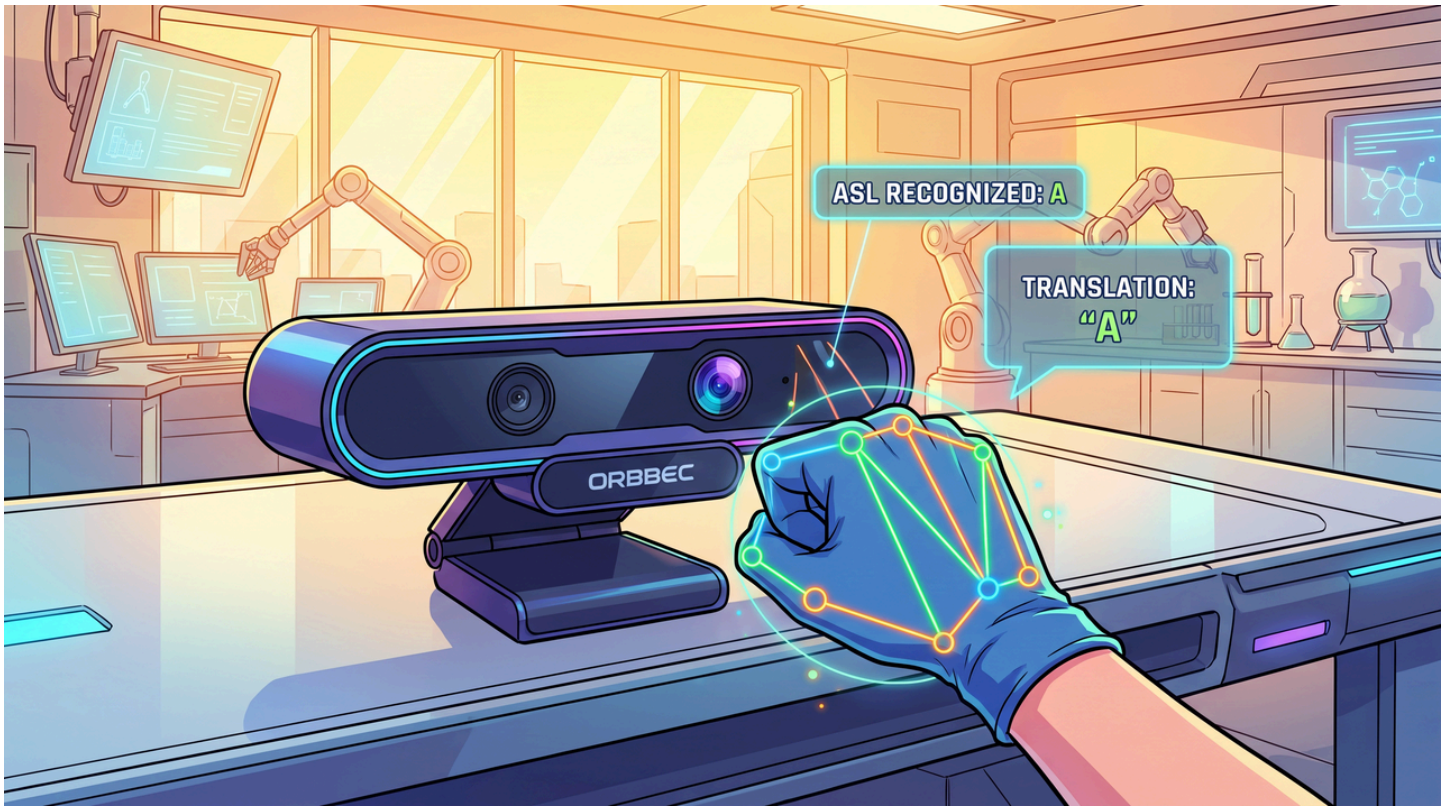
✋ 从深度相机到实时翻译：复现 ASL 美式手语识别与翻译系统



> 本文记录一次基于深圳大学暑期实训营项目「基于 Femto Mega 深度相机开发的美式手语识别与翻译系统」的本地复现过程。

> 原项目面向 Orbbec Femto Mega 深度相机开发；本文实际使用 **Orbbec Gemini 335L** 进行复现。由于二者同属 Orbbec RGB-D 深度相机生态，核心思路一致，但 SDK、深度流初始化和设备调试部分需要特别注意。

1. 项目简介

该系统的目标是将摄像头捕获到的美式手语指拼动作实时识别为英文字母，并进一步拼接为英文单词与句子。

核心功能包括：

- 🧤 ASL 静态字母识别
- ✍️ J / Z 动态手势轨迹识别
- 🔤 字母拼词与拼写纠错
- 🧠 可选 DeepSeek LLM 句子整理
- 🎭 基于人脸识别的用户登录
- 🧩 用户快捷短语系统
- 🌐 FastAPI + WebSocket 实时网页端展示
- 📏 RGB-D 深度信息辅助距离提示与活体检测

项目目录如下：

代码块

```
ASL-Recognition/
├── backend/
│   ├── main.py
│   ├── camera_manager.py
│   ├── face_auth.py
│   ├── shortcuts_manager.py
│   └── pipelines/
│       └── hand_asl.py
├── frontend/
│   ├── index.html
│   └── letters/
├── models/
│   ├── hand_landmarker.task
│   └── model5.keras
├── docs/
├── requirements.txt
└── start.bat
```

2. 复现环境

硬件

项目	本次复现配置
深度相机	Orbbec Gemini 335L
电脑系统	Windows 10 / Windows 11 均可
Python	推荐 Python 3.10 或 3.11
浏览器	Chrome / Edge
网络	如需手机访问，电脑与手机需处于同一局域网

> 注意：项目原始说明中提到 Femto Mega；本文使用 Gemini 335L 复现时，仍然走 Orbbec SDK 的 RGB-D 采集逻辑。

软件依赖

基础依赖来自 `requirements.txt`：

代码块

```
fastapi
uvicorn
websockets
keras
tensorflow
opencv-python
mediapipe
openai
pyspellchecker
numpy
deepface
tf-keras
scipy
```

若使用 Gemini 335L 的真实深度流，额外安装：

代码块

```
pip install --upgrade pyorbbecsdk2
```

截至 2026-08-05，Orbbec 官方 Python SDK v2 文档推荐安装包名为 `pyorbbecsdk2`，但代码中常见导入模块名仍可能是 `pyorbbecsdk`。可参考：

- Orbbec SDK v2: https://github.com/orbbec/OrbbecSDK_v2
- pyorbbecsdk2: https://pypi.org/project/pyorbbecsdk2/
- Orbbec Python Wrapper 文档: https://orbbec.github.io/pyorbbecsdk/source/2_installation/install_the_package.html

3. 安装与启动步骤

3.1 创建虚拟环境

代码块

```
cd D:\Femto_mega\ASL-Recognition

python -m venv venv
.\venv\Scripts\activate
```

3.2 安装项目依赖

代码块

```
pip install -r requirements.txt
```

如果需要启用 Gemini 335L 深度相机：

代码块

```
pip install --upgrade pyorbbecsdk2
```

安装后可验证：

代码块

```
python -c "import pyorbbecsdk; print('Orbbec SDK import ok')"
```

3.3 配置环境变量

只运行普通识别功能时，可以不配置任何环境变量。

如需启用 Orbbec 深度相机：

代码块

```
$env:ENABLE_ORBBEC = "1"
```

如需启用 DeepSeek LLM 句子整理：

代码块

```
$env:DEEPSEEK_API_KEY = "sk-xxxxxxxxxxxxxxxx"
$env:DEEPSEEK_BASE_URL = "https://api.deepseek.com"
```

3.4 启动服务

方式一：Windows 一键启动

```
start.bat
```

方式二：手动启动

```
.\venv\Scripts\python -m uvicorn backend.main:app --host 0.0.0.0 --port 8765
```

启动后访问：

```
http://127.0.0.1:8765
```

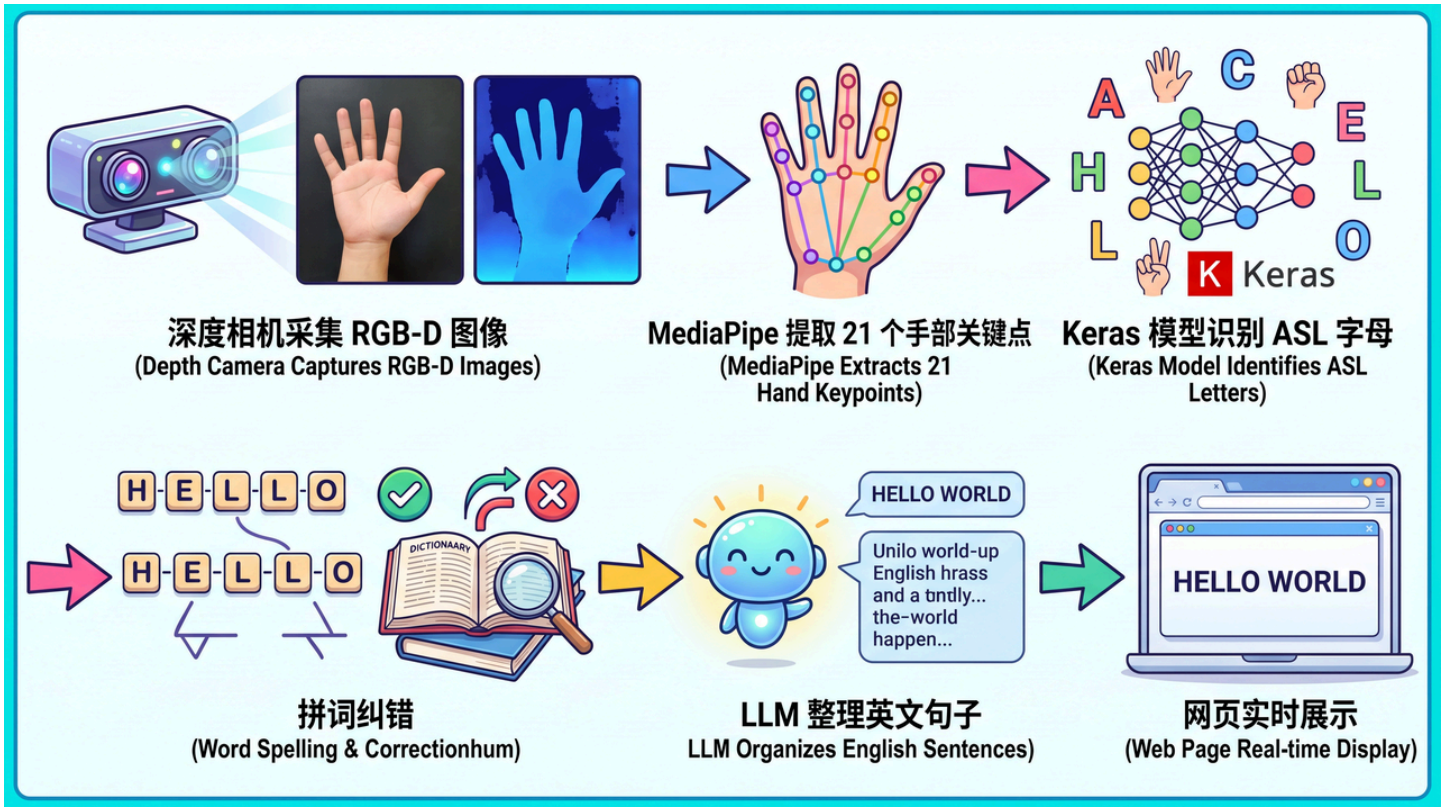
若需要局域网访问，使用终端打印出的地址：

```
http://<电脑局域网 IP>:8765
```

4. 运行流程

1. 启动后端服务。
2. 浏览器打开网页端。
3. 后端调用电脑连接的摄像头采集视频帧。
4. 页面通过 WebSocket 接收实时视频流、识别结果和手部关键点。
5. 在摄像头前比出 ASL 字母。
6. 系统稳定识别后提交字母。
7. 停顿一段时间后自动断词，并进行拼写纠错。
8. 如果配置了 DeepSeek API Key，则进一步整理为自然英文句子。

> 一个容易误解的地方：该项目不是由浏览器直接调用本机摄像头，而是由 Python 后端采集摄像头，再通过 WebSocket 推送到前端。因此访问页面的手机或其他电脑并不需要连接摄像头。



5. 关键技术实现解析

5.1 相机采集：OpenCV 与 Orbbec 自动切换

源码位置：`backend/camera_manager.py`

项目默认使用 OpenCV 摄像头。如果设置了 `ENABLE_ORBBEC=1`，则尝试初始化 Orbbec RGB-D 相机；失败后自动回退到普通 OpenCV 模式。

代码块

```
class CameraManager:
    def __init__(self):
        self.mode = "OPENCV"
        self.pipeline = None
        self.cap = None
        self._init_camera()

    def _init_camera(self):
        has_ob = False
        if os.environ.get("ENABLE_ORBBEC", "").strip() == "1":
            try:
                import pyorbbecsdk2 as ob
                has_ob = True
                self.ob = ob
            except ImportError:
                try:
                    from pyorbbecsdk import Pipeline, Config, OBSensorType,
OBFormat
                    import pyorbbecsdk as ob
                    has_ob = True
                    self.ob = ob
                except ImportError:
                    pass

        if has_ob:
            try:
                self.pipeline = self.ob.Pipeline()
                config = self.ob.Config()
                config.enable_stream(self.ob.OBSensorType.COLOR_SENSOR)
                config.enable_stream(self.ob.OBSensorType.DEPTH_SENSOR)
                config.set_align_mode(self.ob.OBAlignMode.HW_MODE)
                self.pipeline.start(config)
                self.mode = "ORBBEC"
                return
            except Exception:
                self.pipeline = None

        self.mode = "OPENCV"
        self.cap = cv2.VideoCapture(0)
```

这段设计非常适合教学复现：

即使暂时没有深度相机，系统仍然可以用普通摄像头运行核心 ASL 识别功能；接入 Gemini 335L 后，则可以进一步使用深度图做距离判断和活体检测。

5.2 手部关键点提取与 ASL 分类

源码位置：`backend/pipelines/hand_asl.py`

系统使用 MediaPipe HandLandmarker 提取 21 个手部关键点，再将二维坐标归一化后送入 Keras 模型。

代码块

```
def process(self, image):
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    result = self.landmarker.detect(mp_image)

    if not result.hand_landmarks:
        return None

    hand = result.hand_landmarks[0]
    raw_pts = np.array([[lm.x, lm.y, lm.z] for lm in hand], dtype=np.float32)

    points = raw_pts.copy()
    min_x, max_x = points[:, 0].min(), points[:, 0].max()
    min_y, max_y = points[:, 1].min(), points[:, 1].max()
    points[:, 0] = (points[:, 0] - min_x) / (max_x - min_x + 1e-8)
    points[:, 1] = (points[:, 1] - min_y) / (max_y - min_y + 1e-8)

    points = np.expand_dims(points, axis=0)
    preds = self.model.predict(points[:, :, :2], verbose=0)

    idx = int(preds[0].argmax())
    prob = float(preds[0][idx])
    letter = LETTERS[idx]
    letter = self._fix(letter, raw_pts)

```

```
26        return {"letter": letter, "conf": prob, "landmarks": lms}
```

这里的关键点在于“归一化”。不同人的手大小、距离镜头远近不同，如果直接使用原始坐标，模型泛化能力会变差。代码通过手部包围盒将坐标缩放到统一范围，使模型更关注手势形状本身。

5.3 稳定帧机制：减少单帧抖动

源码位置：`backend/main.py`

代码块

```
1   STABLE_FRAMES = 15
2   RESET_GAP = 20
3   WORD_TIMEOUT = 90
4   MIN_CONFIDENCE = 0.80
5
6   if letter == stable_letter:
7       stable_count += 1
8       jitter_count = 0
9   else:
10      if stable_letter is None:
11          stable_letter = letter
12          stable_count = 1
13          jitter_count = 0
14      else:
15          jitter_count += 1
16          if jitter_count >= 5:
17              stable_letter = letter
18              stable_count = 1
19              jitter_count = 0
20
21  if stable_count >= STABLE_FRAMES and letter != last_committed:
22      raw_word += letter
23      last_committed = letter
```

实时识别时，模型每一帧都会输出预测结果。手部轻微抖动、光照变化、关键点误差都会导致单帧结果不稳定。

因此系统要求连续多帧识别为同一个字母，才真正提交该字母。这样会牺牲一点响应速度，但显著提升可用性。

5.4 J / Z 动态手势识别

ASL 中 J 和 Z 不是纯静态手势，需要根据手指运动轨迹判断。

代码块

```
DYNAMIC_WINDOW_FRAMES = 45
COOLDOWN_FRAMES = 15
dynamic_state = None
dynamic_target = ""

def _check_finger_motion(start_lms, cur_lms, tip_idx, threshold=0.008, ratio=1.5):
    target_d = math.sqrt(
        (cur_lms[tip_idx][0] - start_lms[tip_idx][0]) ** 2 +
        (cur_lms[tip_idx][1] - start_lms[tip_idx][1]) ** 2
    )
    if target_d < threshold:
        return False

    max_other = max(
        math.sqrt(
            (cur_lms[i][0] - start_lms[i][0]) ** 2 +
            (cur_lms[i][1] - start_lms[i][1]) ** 2
        )
        for i in range(21) if i != tip_idx
    )

    return target_d > max_other * ratio
```

项目中的设计思路是：

- 当系统稳定识别到 `I` 时，进入 J 的动态追踪。
- 当系统稳定识别到 `D` 时，进入 Z 的动态追踪。
- J 主要观察小指指尖运动。
- Z 主要观察食指指尖运动。
- 设置冷却帧，避免同一个动态动作被重复提交。

5.5 拼写纠错与 LLM 句子整理

源码位置：`backend/main.py`

代码块

```python
spell = SpellChecker()

def spell_correct(word):
    if not word or len(word) < 2:
        return None

    candidates = spell.candidates(word)
    if candidates:
        return list(candidates)[0]

    active = {i: CONFUSABLES.get(c, []) for i, c in enumerate(word)}
    for i, alts in active.items():
        for alt in alts:
            c = spell.candidates(word[:i] + alt + word[i + 1:])
            if c:
                return list(c)[0]

    return None
```

这部分首先调用 `pyspellchecker` 做英文单词候选纠错。

如果直接纠错失败，再结合项目中定义的易混字母表进行替换尝试，例如 A/T/S、C/E/O、K/V/U 等。

LLM 整理逻辑如下：

代码块

```
def _get_llm():
    global llm
    if llm is None and os.environ.get("DEEPSEEK_API_KEY"):
        llm = OpenAI(
            api_key=os.environ["DEEPSEEK_API_KEY"],
            base_url=os.environ.get("DEEPSEEK_BASE_URL",
"https://api.deepseek.com"),
        )
    return llm

def _llm_format(words, t_id):
    joined = " ".join(words)
    prompt = f"Convert to natural English sentence: {joined}. Output only the
sentence."

    client = _get_llm()
    if client is None:
        formatted_sentence = joined + "..."
        return

    resp = client.chat.completions.create(
        model="deepseek-v4-flash",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=50,
        temperature=0.3,
    )
```

这里采用“可选增强”的方式：

没有 API Key 时，系统仍可正常完成字母识别和单词拼写；配置 API Key 后，才启用自然语言整理。

5.6 WebSocket 实时通信

后端：

代码块

```
@app.websocket("/ws")
```

```python
async def ws_endpoint(ws: WebSocket):
    await ws.accept()

    async def send_loop():
        while True:
            await ws.send_json(current_state)

            latest = None
            while True:
                try:
                    latest = frame_queue.get_nowait()
                except queue.Empty:
                    break

            if latest is not None:
                await ws.send_bytes(latest)

            await asyncio.sleep(0.025)
```

前端：

代码块

```javascript
const ws = new WebSocket(
  (location.protocol === "https:" ? "wss://" : "ws://") + location.host + "/ws"
);

ws.binaryType = "arraybuffer";
```

后端持续推送两类数据：

- JSON：识别结果、置信度、单词、句子、距离警告等状态。
- 二进制 JPEG：实时视频帧。

这种设计比简单 HTTP 轮询更适合实时识别场景。

5.7 深度活体检测

源码位置：`backend/face_auth.py`

代码块

```python
def verify_liveness(depth_frame, facial_area, rgb_h=None, rgb_w=None):
    if depth_frame is None:
        return "bypassed"

    roi = depth_frame[fy1:fy2, fx1:fx2].astype(np.float32)
    valid_all = roi[roi > 0]
    if valid_all.size == 0:
        return "invalid_depth"

    median = float(np.median(valid_all))
    close = roi[(roi > 0) & (roi >= median - 30.0) & (roi <= median + 30.0)]

    std_mm = float(np.std(close))
    if std_mm < LIVENESS_STD_THRESHOLD:
        return "spoof"

    return "genuine"
```

其核心思想是：真实人脸在深度图中应具有一定立体起伏，而照片或屏幕通常更接近平面。

因此代码会统计人脸区域深度值的标准差，若过低则可能判定为伪造输入。

6. Gemini 335L 复现避坑点



6.1 先用 Orbbec Viewer 验证硬件

在运行项目之前，建议先打开 Orbbec Viewer，确认 Gemini 335L 能正常输出：

- Color Stream
- Depth Stream
- 深度和彩色画面对齐是否正常

如果 Viewer 中都无法正常显示，项目代码通常也无法读取到深度帧。

6.2 一定要设置`ENABLE_ORBBEC=1`

如果忘记设置：

代码块

```
$env:ENABLE_ORBBEC = "1"
```

系统会进入 OpenCV 模式。此时 ASL 识别仍可运行，但：

- 没有真实深度图
- 人脸活体检测会降级
- 距离提示只能基于手掌像素大小估算

6.3 浏览器端看到的是后端相机画面

本项目的视频不是浏览器直接采集，而是后端采集后推送到前端。

所以手机访问局域网页面时，看到的仍然是连接在电脑上的 Gemini 335L / USB 摄像头画面。

6.4 TensorFlow 版本可能是最大安装障碍

建议优先使用 Python 3.10 或 3.11。

如果安装 TensorFlow 报错，可尝试：

```
代码块
1   pip install tensorflow-cpu
```

6.5 摄像头被占用会导致黑屏

如果微信、腾讯会议、相机应用正在占用摄像头，OpenCV 或 Orbbec SDK 可能无法读取帧。

复现前建议关闭所有可能占用摄像头的软件。

6.6 J / Z 动作要慢一点、幅度大一点

动态手势依赖轨迹窗口判断。动作太快、太小或手指没有明显移动时，容易被识别成 I 或 D。

7. 推荐演示顺序

建议按以下顺序展示项目，观众更容易理解：

1. 打开网页端，说明这是实时 Web 系统。
2. 展示摄像头画面和手部关键点骨架。
3. 比出几个静态字母，例如 A、B、C、L。
4. 拼一个简单单词，例如 `HELP`。
5. 停顿，展示自动断词和拼写纠错。
6. 展示 J / Z 动态手势。
7. 如果配置了 DeepSeek API Key，展示句子整理。
8. 如果 Gemini 335L 深度流正常，展示人脸登录和活体检测。

8. 总结

本项目的价值不只在于“识别出字母”，更在于它把多个工程模块串成了一个完整的交互系统：

- 视觉感知：MediaPipe 手部关键点
- 模型推理：Keras ASL 分类模型
- 稳定性工程：连续帧投票、防抖、冷却机制
- 语言后处理：拼写纠错与 LLM 句子整理
- 人机交互：WebSocket 实时网页端
- 深度相机能力：距离估计与活体检测

使用 Gemini 335L 复现时，核心 ASL 识别流程可以顺利迁移；主要差异集中在 Orbbec SDK 安装、深度流读取、环境变量配置和相机调试上。整体来看，这是一个很适合作为暑期实训、课程设计或计算机视觉入门项目的综合案例。🌟